HKPUG Meetup | June 2026

Building a Spatial Vision Framework Completely Alone Handcrafting L.E.P.A.U.T.E. Geometry & Asynchronous Code

A solo developer's journey hacking simulators, pipeline architecture, and Lie groups

Speaker: Carson Wu
CV & NLP Developer

Hong Kong Python User Group
27 June 2026

Speaker Information & Project Repository

Scan to Connect

L.E.P.A.U.T.E. Repository

Part I: What is a Lie Group?

Let's address the elephant in the room. Why do mathematicians use fancy words like **Lie Groups** when they just mean moving things around?

- **The Intuition:** A Lie group is a group that is also a **smooth manifold**. It represents a continuous space of symmetries or transformations.
- **Our Core Focus:** For spatial vision, we care about

$$\mathrm{SE}(3)$$

(Special Euclidean Group). It encompasses all continuous 3D rotations and translations.

- **The Real-World Reality:** When a car bumps, pitches, or rolls, its change in camera perspective is a continuous journey along this smooth geometric surface.

Why Normal Neural Networks Hate Raw Matrices

Why can't we just dump a standard 4×4 transformation matrix into a linear layer and let the network figure it out?

The Non-Euclidean Trap:
If your neural network updates a 4×4 matrix via standard backpropagation, it will quickly distort the upper-left 3×3 matrix. Suddenly, your rotation matrix scales or shears space. Your virtual camera is now an abstract painting.

- **Euler Angles:** Suffer from **Gimbal Lock** and wrap-around discontinuities (180° suddenly flipping to −180°).

- **The Mathematical Cure:** We need a way to parameterize coordinates linearly without ever escaping the geometric boundaries of true physical space.

The Tangent Space: Introducing Lie Algebra

Since deep learning engines love flat, linear vector coordinates, we map the curved manifold surface onto its local flat tangent space at the Identity matrix.

- **The Lie Algebra**

$$\mathfrak{se}(3)$$

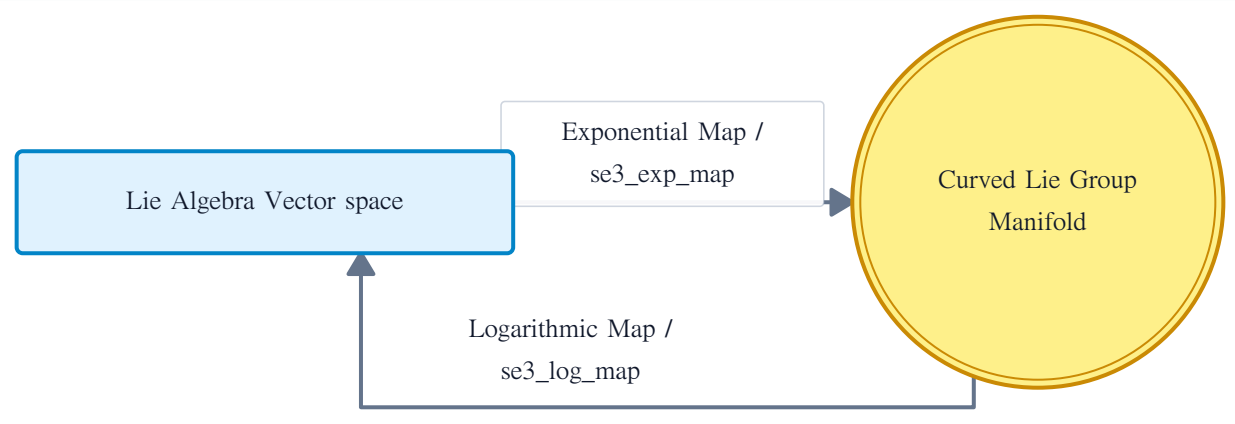
  : This acts as our local flat vector space.
- Any 3D velocity or relative twist can be fully represented as a simple 6-element vector:

$$\xi = (v, \omega) \in \mathfrak{se}(3)$$

- **The Bridge:** We map back and forth seamlessly using exact calculus conversions:

- **Exponential Map** (se3_exp_map): Maps a flat 6D vector up onto the true curved 3D transformation manifold.
- **Logarithmic Map** (se3_log_map): Flattens a 3D transformation matrix down into 6 linear numbers.

Visualizing the Exponential and Logarithmic Maps



By keeping our parameters in

$$\mathfrak{se}(3)$$

during neural processing, our model can optimize relative pose changes smoothly using standard optimization algorithms without ever corrupting the intrinsic laws of geometry.

Part II: L.E.P.A.U.T.E. Core Technical Architecture

Most autonomous vision frameworks are massively heavy. As a solo dev, I don't have a massive cluster to train giant models on millions of miles, so I thought: **Can we use geometric laws to force the network to take a shortcut?**

My core design principles to make this possible:

- **Physics instead of memory:** Map continuous sequence frames directly onto

$$SE(3)$$

  Lie groups, so spatial awareness is baked into the math.
- **Enforcing consistency:** Ensure that when the vehicle pitches or rolls in the simulator, the extracted features don't lose their spatial orientation.

- **Stitching new tools together:** Along with optical flow, I threw in Transformer structural embeddings and a Zero-shot Open-Vocabulary classifier to see how far I could push it.

Solo Architecture: Keeping things alive by breaking threads

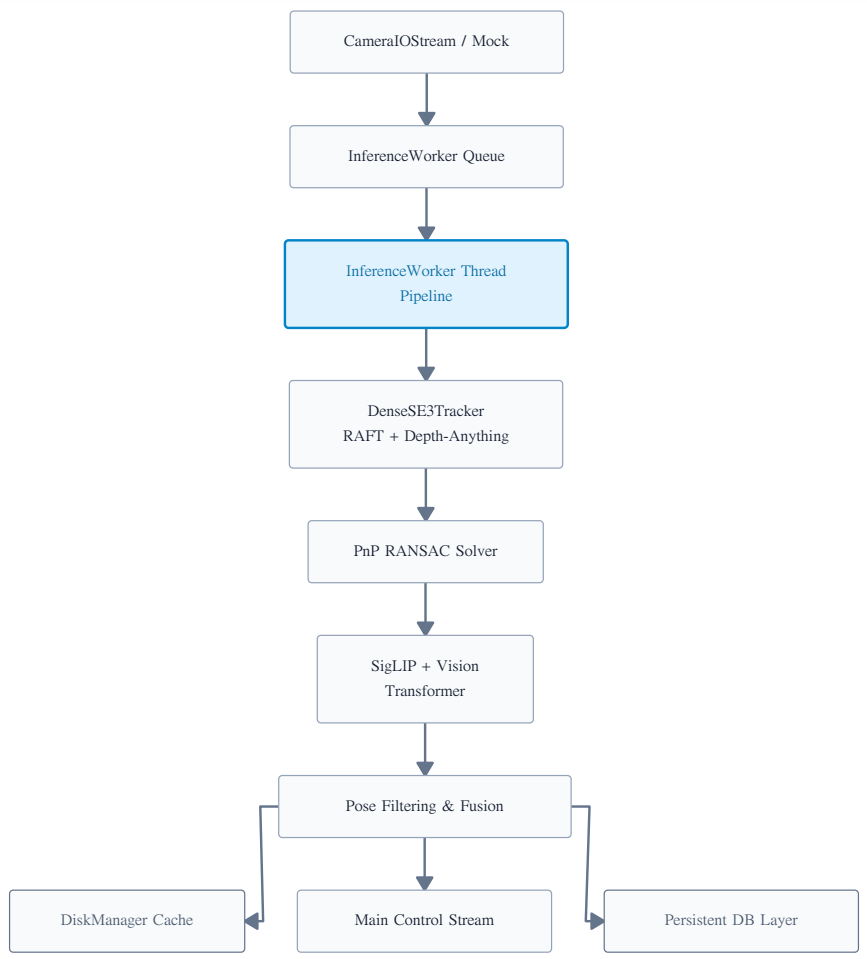
Full disclosure: **This framework hasn't touched a real vehicle yet.** But testing on my desktop simulator setup taught me early on that mixing ingestion with inference breaks everything immediately.

- **Isolated Ingestion Ring:** Image capture runs on its own isolated loop. No matter how much the downstream models lag, the camera/simulator stream never drops a frame.
- **Asynchronous Inference Pool:** Heavyweights like Depth-Anything and SigLIP are thrown into separate worker threads to crunch data at their own pace.

- **Zero-Overhead Persistence:** Atomic operations dump telemetry directly into caches and local SQLite files so I can review exactly where my tracking drifted post-run.

How the Pipeline Loops in the Background

This is the asynchronous topology running locally on my development machine:


CameraIOStream / Mock
InferenceWorker Queue
InferenceWorker Thread
Pipeline
DenseSE3Tracker
RAFT + Depth-Anything
PnP RANSAC Solver
SigLIP + Vision
Transformer
Pose Filtering & Fusion
DiskManager Cache
Main Control Stream
Persistent DB Layer

Hardcore Geometry: Why bother with SE(3)?

Without geometric constraints, if your vehicle hits a massive bump in a virtual world and shakes violently, the model completely loses its bearings. I wanted my code to have an innate “sense of balance.”

Lie Algebra Equivariance Constraint:

When a physical movement warps the input image sequence by some transformation

$$g \in \mathrm{SE}(3)$$

, the feature extractor $f(x)$ naturally flows equivariantly with it:

$$f(g \cdot x) = g \cdot f(x)$$

- **The real-world benefit:** Forcing this mathematical constraint straight into the tensor layers gives the system geometric memory, keeping tracking stable even during erratic movement.

The Joy of Custom Code: Differentiable Warping & Rings

1. Differentiable Warping Layers

- Implemented inside module.py as MonocularSE3Warping.
- It projects pixels into 3D coordinates using depth scale priors, applies the

$$\mathfrak{se}(3)$$

twist, and samples the next frame via a fully differentiable grid_sample operator.

2. Squeezing Latency with Buffers

- Designed an InferenceWorker queue thread loop inside main.py.
- It uses thread-safe locks to isolate inference from capture loops.
- Discards obsolete frames seamlessly under high loads, ensuring the main trajectory fusion system always reads the fresh spatial updates.

- Enables direct gradient propagation from pixel discrepancies back into estimated geometric motion.

Open-Vocabulary and Multi-Tasking (Where I lost my hair)

Handling Unexpected Obstacles

- I plugged SigLIP's zero-shot classification embeddings straight into the perception pipeline inside module.py.
- If an odd object, like a strange cardboard box or fallen debris, appears in the simulator, I don't have to retrain. I just pass a new text label string at runtime to evaluate its geometric scale prior.

The Balancing Act of Multi-Loss

- This was my absolute biggest pain point: ego-pose tracking requires heteroscedastic uncertainty parameter optimization.
- Balancing the visual spatial reconstruction against regression variance felt like walking a tightrope, but after countless nights, they finally converged together.

Defensive Programming: Handling Failures at Home

- **Sensor Disconnect Recovery:** If a webcam drops or the simulator's webhook hits a lag spike, the ingestion layer seamlessly invokes ManifoldKinematicForecaster. It extrapolates motion using median historic velocity history entirely on the

$$\mathfrak{se}(3)$$

  manifold to prevent system execution crashes.
- **Database & Memory Protection:** Long testing sessions generate massive telemetry tracking pairs. The pipeline leverages asynchronous SQLite WAL logging via SequenceDataCollector to write state packets without starving processing cycles.

What This Taught Me in Virtual Worlds

Even though this code hasn't seen a single mile of asphalt yet, the results on my testbench show massive potential for solo developers:

- **Math scales better than data:** It proves you don't need a massive team throwing datasets at a wall. Strict geometric priors allow lightweight setups to excel in spatial mapping.
- **Isolation brings stability:** Separating the threads makes the pipeline incredibly robust against latency spikes and hardware stuttering.
- **Dynamic adaptability:** Open-vocabulary setups make it trivial to introduce new object classes on the fly without running heavy training loops.

How to Run It: Runtime Configurations

Dependencies are isolated minimal requirements listed cleanly in requirements.txt. When deploying this on my local PC or a standalone testboard, I use simple environment flags to keep configuration overhead to zero:

```
export LEPAUTE_DEVICE="cuda"
export LEPAUTE_MAX_DISK_FILES=2000
export LEPAUTE_MODE="autonomous"
```

If you want to look at the raw matrix values spitting out of the geometric layers in real time, simply drop the logging level down to Debug.

Code Execution: Three Lines to Fire Up the Engine

The underlying multithreading code and C++ geometric wrappers took months to figure out, but the final Python API exported by __init__.py is dead simple:

```
import logging
from main import run_pipeline
from module import LepauteConfig, DisplayMode

logging.basicConfig(level=logging.INFO)

config = LepauteConfig(device="cuda")
processed_data = run_pipeline(
    config=config,
    display_mode=DisplayMode.JSON,
    mock=True
)
```

Offline training and data formatting routines can be easily evaluated using convert_bop_to_lepaute.py and example.py. Verification checks can be systematically run via test_module.py.

Live Demo

Real-time Spatial Vision Pipeline

Watch the L.E.P.A.U.T.E. framework running live
with simulator input and SE(3) pose tracking

Q&A

Feel free to ask about the math, architecture, or threading failures!

Share This Keynote Presentation

Scan to download keynote and abstract materials

Thank You!